ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Piotr Nieużyła

Klasa: 5P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Wstęp ... **3**
Wprowadzenie do tematu testowania ... 3
Podstawowe pojęcia (terminologia ISTQB) ... 3
Typy i poziomy testów ... 3
Podstawowe zasady testowania ... 3
Proces testowy ... 4
1. Testy jednostkowe ... **4**
Definicja i cel wykonywania testów jednostkowych ... 4
Przykład opisowy (bez kodu) ... 4
Opis narzędzia do testowania – xUnit ... 4
Wzorzec AAA (Arrange – Act – Assert) ... 5
1.1 Implementacja programu ... 5
1.2 Implementacja testów ... 5
1.3 Git ... 6
2. Testy integracyjne ... **7**
Definicja i cel wykonywania testów integracyjnych ... 7
2.1 Implementacja programu (Integracja) ... 7
2.2 Implementacja testów integracyjnych ... 8
2.3 Git ... 9

Wstęp

Wprowadzenie do tematu testowania

Testowanie w cyklu SDLC (*Software Development Life Cycle*) to proces weryfikacji i walidacji oprogramowania. Jego celem jest potwierdzenie zgodności systemu z wymaganiami funkcjonalnymi i niefunkcjonalnymi oraz wykrycie defektów przed przekazaniem aplikacji do użytku.

Podstawowe pojęcia (terminologia ISTQB)

- **Błąd / Pomyłka (Error):** Działanie człowieka (np. programisty) prowadzące do powstania nieprawidłowego rezultatu w kodzie.
- Defekt / Usterka (Bug): Wada w kodzie lub dokumentacji będąca konsekwencją pomyłki.
- Awaria (Failure): Nieprawidłowe zachowanie systemu podczas uruchomienia, wywołane wystąpieniem defektu.

Typy i poziomy testów

1. **Testy jednostkowe (Unit Testing):** Weryfikacja pojedynczych, odizolowanych elementów kodu (metod, klas).
2. Testy integracyjne (Integration Testing): Sprawdzanie współdziałania połączonych modułów.
3. Testy systemowe (System Testing): Kompleksowa weryfikacja całej zintegrowanej aplikacji.
4. Testy akceptacyjne (Acceptance Testing): Ostateczna weryfikacja systemu przeprowadzana z perspektywy użytkownika końcowego.

Podstawowe zasady testowania

1. Testowanie ujawnia obecność błędów, a nie ich brak.
2. Testowanie wyczerpujące jest niemożliwe.
3. Wczesne testowanie oszczędza czas i koszty.
4. Defekty kumulują się w niewielkiej liczbie modułów.
5. Paradoks pestycydów (testy wymagają regularnej aktualizacji).

6. Testowanie zależy od kontekstu.
7. Przekonanie o braku błędów jest błędem.

Proces testowy

1. Planowanie i kontrola.
2. Analiza i projektowanie testów.
3. Implementacja i wykonanie.
4. Ocena kryteriów zakończenia i raportowanie.
5. Zamknięcie testów.

1. Testy jednostkowe

Definicja i cel wykonywania testów jednostkowych

Testy jednostkowe to automatyczne testy weryfikujące najmniejsze, odizolowane elementy kodu źródłowego (metody, klasy). Służą do szybkiego wykrywania błędów na wczesnym etapie prac oraz ułatwiają bezpieczny refaktoryzację kodu.

Przykład opisowy (bez kodu)

- **Test pozytywny:** Koszyk o wartości 100 zł z ważnym kodem -10% → metoda zwraca kwotę 90 zł.
- **Test negatywny:** Koszyk o wartości 100 zł z wygasłym kodem → metoda wyrzuca wyjątek/błąd walidacji, a kwota pozostaje 100 zł.

Opis narzędzia do testowania – xUnit

xUnit.net to framework testowy dla platformy .NET (C#). Kluczowe elementy:

- `[Fact]` – definiuje pojedynczy, bezparametryczny przypadek testowy.
- [Theory] – definiuje test sparametryzowany, uruchamiany dla wielu zestawów danych (np. z [InlineData]).
- Asercje (Assert) – metody weryfikujące wyniki:
    - Assert.Equal(expected, actual) – sprawdza równość wartości.
    - Assert.NotEqual(expected, actual) – sprawdza nierówność wartości.
    - Assert.Throws<TException>(...) – weryfikuje rzucenie oczekiwanego wyjątku.

Wzorzec AAA (Arrange – Act – Assert)

1. **Arrange:** Przygotowanie obiektów i danych wejściowych.
2. Act: Wywołanie testowanej metody.
3. Assert: Weryfikacja wyniku za pomocą asercji.

1.1 Implementacja programu

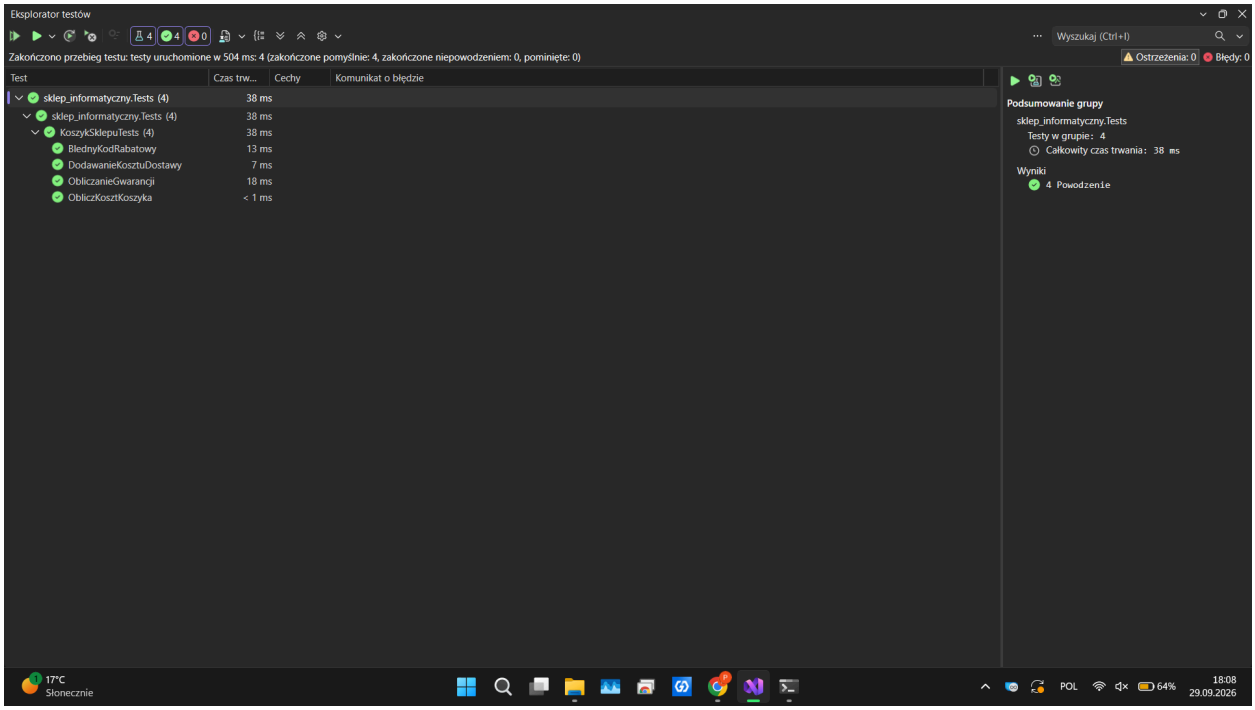
Program symuluje logikę koszyka w sklepie komputerowym. Składa się z klasy reprezentującej produkt (Produkt) oraz klasy zarządzającej koszykiem i naliczaniem opłat (KoszykSklepu).
Główne metody:

- **DodajProdukt** – dodaje przedmiot do koszyka, walidując ujemne ceny i ilości (<= 0).
- **DodajZnizke** – ustawia rabat na podstawie kodu (STUDENT = 15%, PROMO5 = 50%); odrzuca puste i niepoprawne kody.
- **ObliczKoszt** – wylicza łączną kwotę po rabacie, doliczając 15 zł dostawy przy koszyku poniżej 200 zł.
- **ObliczGwarancje** – oblicza koszt rozszerzonej gwarancji (10% ceny za rok, w przedziale 1–5 lat).

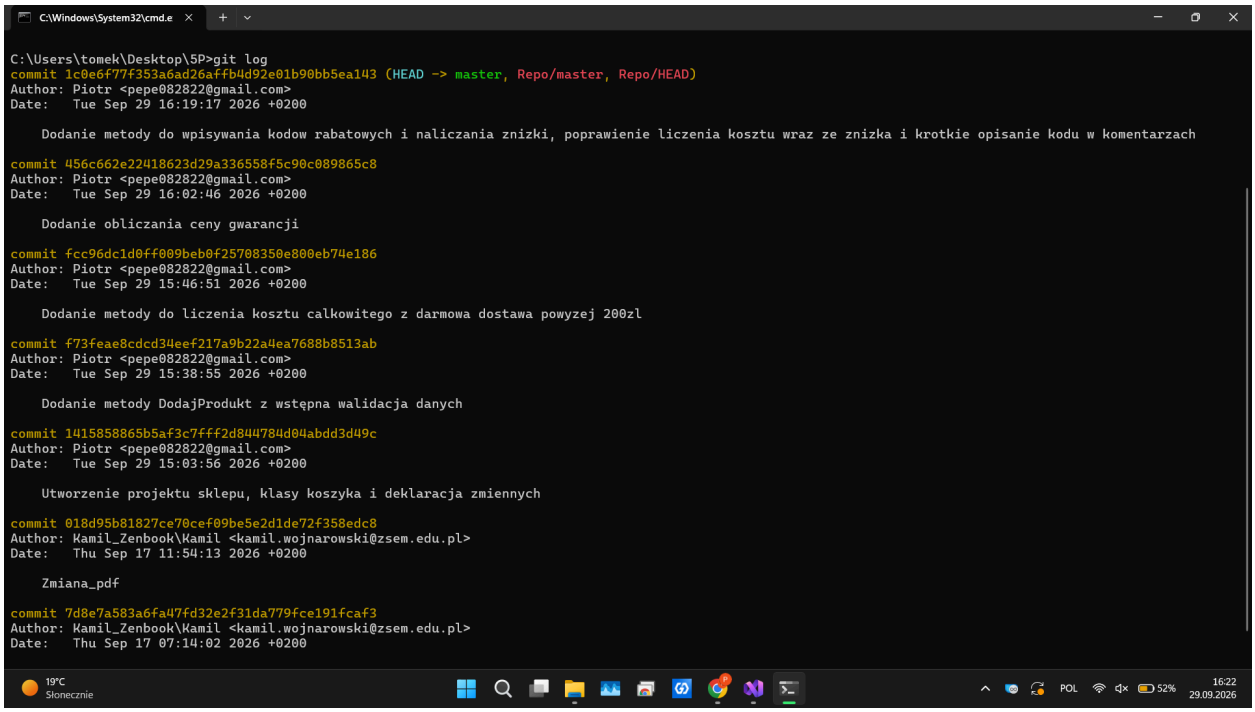
1.2 Implementacja testów

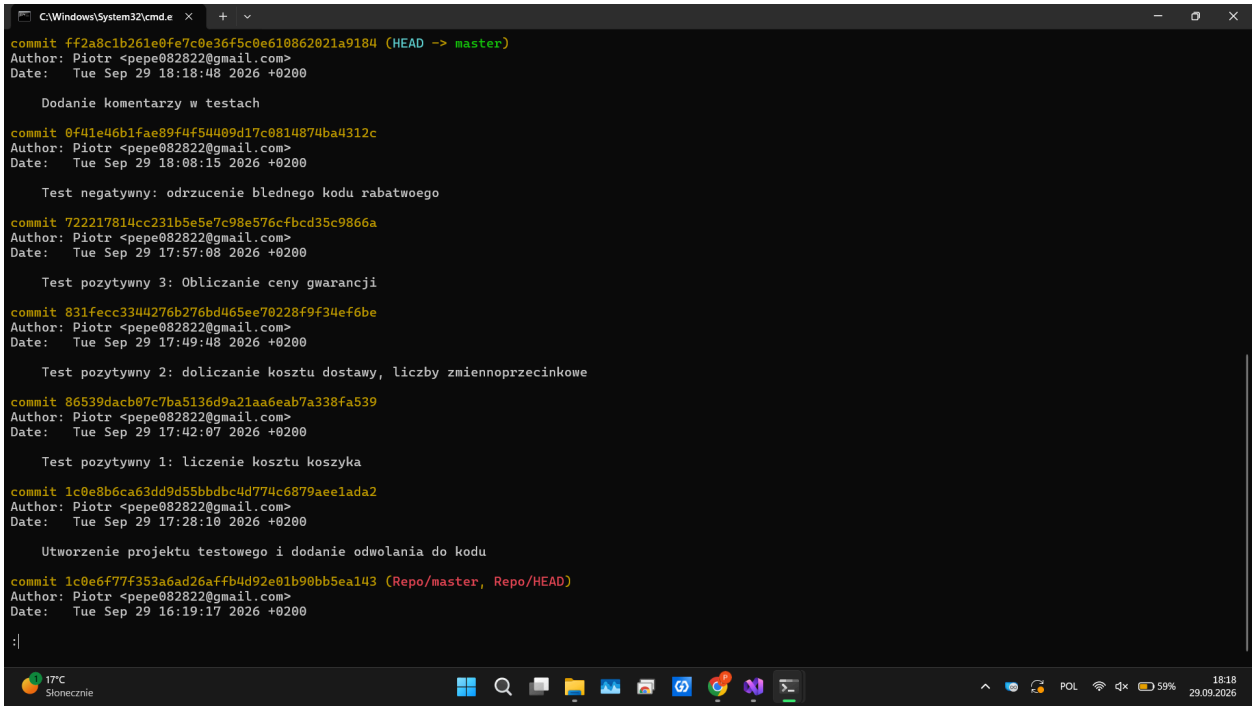
Zestaw testów xUnit weryfikuje poprawność przeliczeń finansowych, naliczanie dostawy oraz walidację błędów w klasie KoszykSklepu.
Przypadki testowe:

- **ObliczKosztKoszyka (Pozytywny)** – sprawdza sumowanie kosztu kilku przedmiotów (oczekiwane: 4500 zł).
- **DodawanieKosztuDostawy (Pozytywny)** – weryfikuje doliczenie 15 zł dostawy przy zakupie poniżej 200 zł (oczekiwane: 80,99 zł).
- **ObliczanieGwarancji (Pozytywny)** – wylicza koszt 3-letniej gwarancji dla kwoty 2000 zł (oczekiwane: 600 zł).
- **BlednyKodRabatowy (Negatywny)** – sprawdza wyrzucenie wyjątku ArgumentException przy podaniu nieprawidłowego kodu.


Eksplorator testów
Wyszukaj (Ctrl+I)
Zakończono przebieg testu: testy uruchomione w 504 ms: 4 (zakończone pomyślnie: 4, zakończone niepowodzeniem: 0, pominięte: 0)
Ostrzeżenia: 0
Błędy: 0
Test
Czas trw...
Cechy
Komunikat o błędzie
sklep_informatyczny.Tests (4) 38 ms
sklep_informatyczny.Tests (4) 38 ms
KoszykSklepuTests (4) 38 ms
BlednyKodRabatowy 13 ms
DodawanieKosztuDostawy 7 ms
ObliczanieGwarancji 18 ms
ObliczKosztKoszyka < 1 ms
Podsumowanie grupy
sklep_informatyczny.Tests
Testy w grupie: 4
Całkowity czas trwania: 38 ms
Wyniki
4 Powodzenie
17°C
Słonecznie
POL
64%
18:08
29.09.2026

1.3 Git


C:\Windows\System32\cmd.e
C:\Users\tomek\Desktop\5P>git log
commit 1c0e6f77f353a6ad26affb4d92e01b90bb5ea143 (HEAD -> master, Repo/master, Repo/HEAD)
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 16:19:17 2026 +0200
Dodanie metody do wpisywania kodow rabatowych i naliczania znizki, poprawienie liczenia kosztu wraz ze znizka i krotkie opisanie kodu w komentarzach
commit 456c662e22418623d29a336558f5c90c089865c8
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 16:02:46 2026 +0200
Dodanie obliczania ceny gwarancji
commit fcc96dc1d0ff009beb0f25708350e800eb74e186
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 15:46:51 2026 +0200
Dodanie metody do liczenia kosztu calkowitego z darmowa dostawa powyzej 200zl
commit f73feae8cdcd34eef217a9b22a4ea7688b8513ab
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 15:38:55 2026 +0200
Dodanie metody DodajProdukt z wstępna walidacja danych
commit 14158588655b5af3c7fff2d844784d04abdd3d49c
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 15:03:56 2026 +0200
Utworzenie projektu sklepu, klasy koszyka i deklaracja zmiennych
commit 018d95b81827ce70cef09be5e2d1de72f358edc8
Author: Kamil_Zenbook\Kamil <kamil.wojnarowski@zsem.edu.pl>
Date: Thu Sep 17 11:54:13 2026 +0200
Zmiana_pdf
commit 7d8e7a583a6fa47fd32e2f31da779fce191fcaf3
Author: Kamil_Zenbook\Kamil <kamil.wojnarowski@zsem.edu.pl>
Date: Thu Sep 17 07:14:02 2026 +0200
19°C
Słonecznie
POL
52%
16:22
29.09.2026



2. Testy integracyjne

Definicja i cel wykonywania testów integracyjnych

Testy integracyjne to kolejny poziom testowania po testach jednostkowych. Ich celem jest sprawdzenie, czy poszczególne moduły lub klasy systemu współpracują ze sobą poprawnie i wymieniają właściwe dane (np. współpraca logiki koszyka ze stanami magazynowymi podzespołów).

2.1 Implementacja programu (Integracja)

Do projektu dodano klasę `MagazynSklepu`, która przechowuje stany magazynowe i ceny podzespołów komputerowych w słowniku. Klasa `KoszykSklepu` współpracuje z magazynem, weryfikując dostępność towarów oraz pobierając ich aktualne ceny przed dodaniem do zamówienia.
Główne metody:

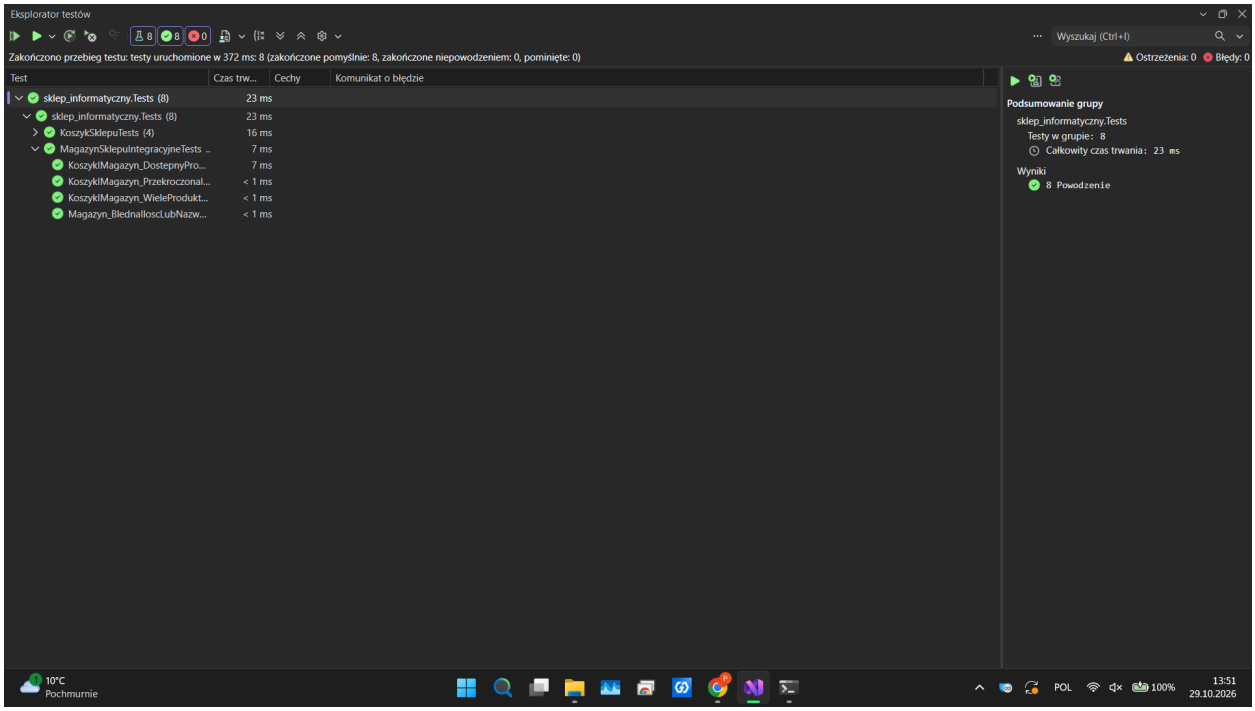
- **SprawdzDostepnosc** – weryfikuje, czy produkt istnieje w magazynie i czy jest wystarczająca liczba sztuk.
- **PobierzCene** – zwraca cenę podzespołu bezpośrednio z magazynu.

2.2 Implementacja testów integracyjnych

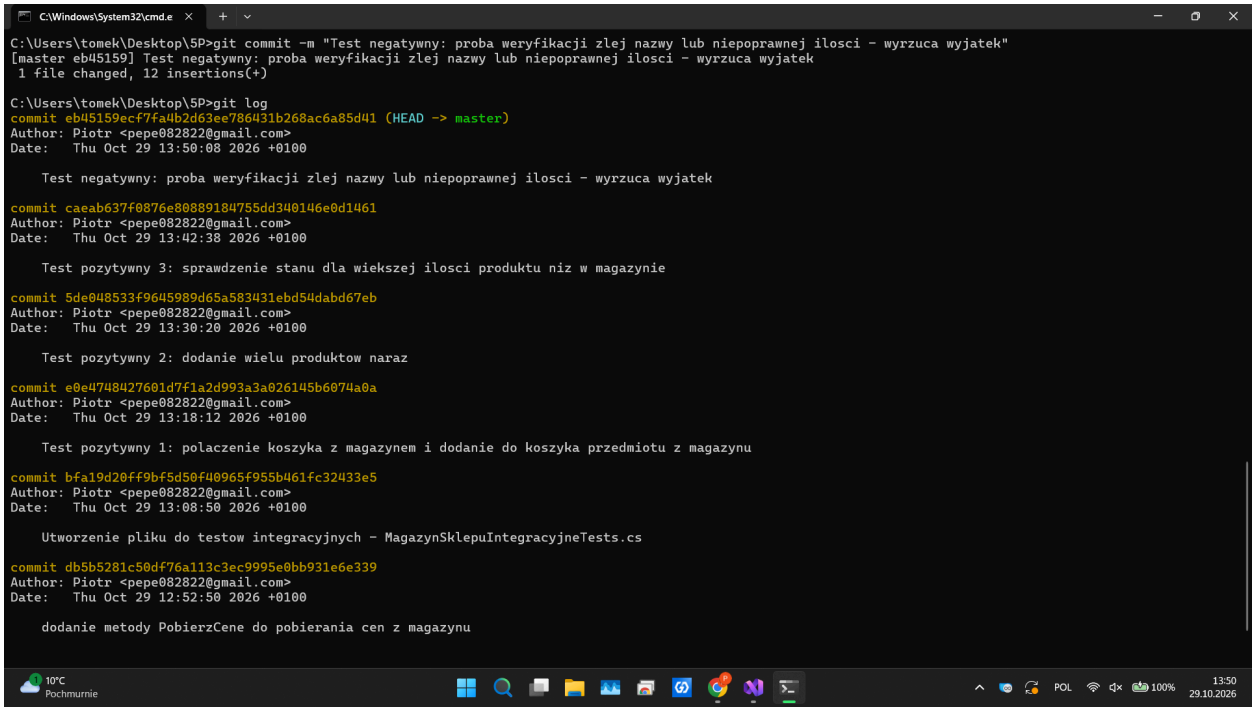
Zestaw testów xUnit w klasie `MagazynSklepuIntegracyjneTests` weryfikuje poprawność współpracy między koszykiem a magazynem.

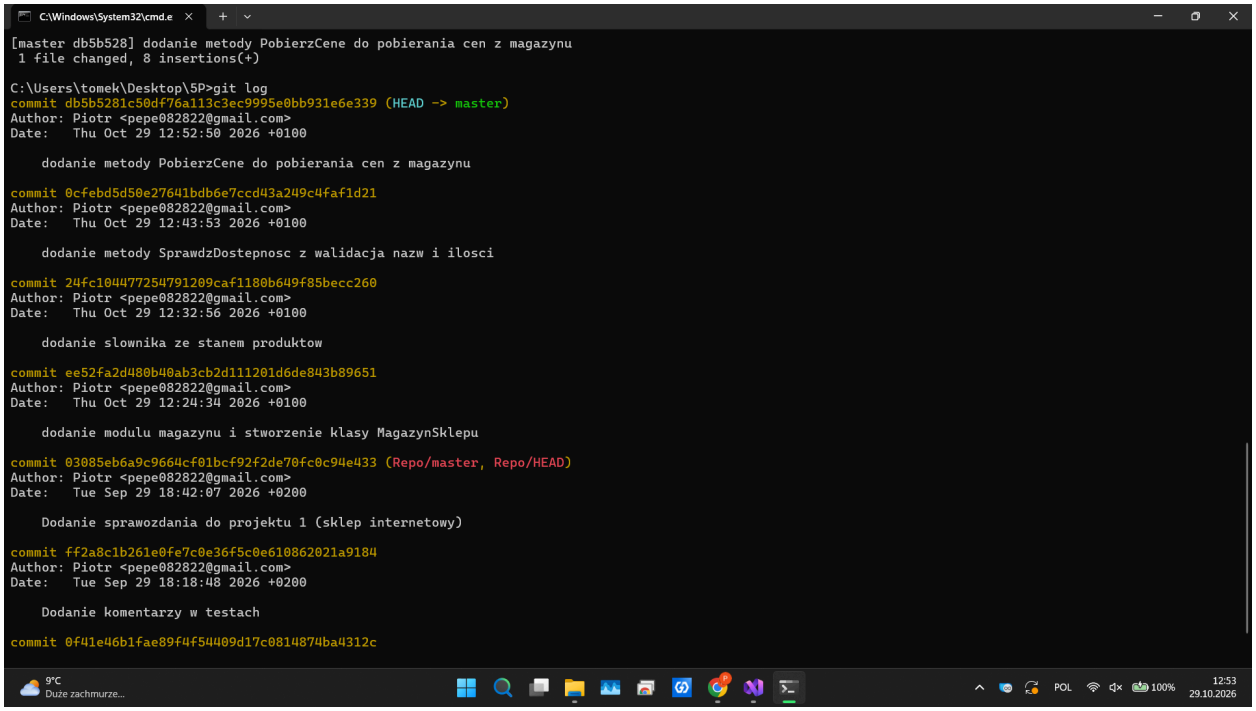
Przypadki testowe:

- **KoszykIMagazyn_DostepnyProdukt_ZwracaPrawdaISumujeCene (Pozytywny)** – sprawdza udany zakup dostępnego podzespołu z pobraniem ceny.
- **KoszykIMagazyn_WieleProduktow_PoprawnaIntegracja (Pozytywny)** – weryfikuje integrację wielu podzespołów komputerowych jednocześnie.
- **KoszykIMagazyn_Przekroczonallosc_OdrzucaZakup (Pozytywny)** – sprawdza odrzucenie zakupu przy przekroczeniu stanów magazynowych.
- **Magazyn_BlednalloscLubNazwa_RzucaWyjatek (Negatywny)** – weryfikuje rzucenie wyjątku `ArgumentException` przy błędnych danych wejściowych.



2.3 Git


C:\Windows\System32\cmd.e
C:\Users\tomek\Desktop\5P>git commit -m "Test negatywny: proba weryfikacji zlej nazwy lub niepoprawnej ilosci - wyrzuca wyjatek"
[master eb45159] Test negatywny: proba weryfikacji zlej nazwy lub niepoprawnej ilosci - wyrzuca wyjatek
1 file changed, 12 insertions(+)
C:\Users\tomek\Desktop\5P>git log
commit eb45159ecf7fa4b2d63ee786431b268ac6a85d41 (HEAD -> master)
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 13:50:08 2026 +0100
Test negatywny: proba weryfikacji zlej nazwy lub niepoprawnej ilosci - wyrzuca wyjatek
commit caeab637f0876e80889184755dd340146e0d1461
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 13:42:38 2026 +0100
Test pozytywny 3: sprawdzenie stanu dla wiekszej ilosci produktu niz w magazynie
commit 5de048533f9645989d65a583431ebd54dabd67eb
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 13:30:20 2026 +0100
Test pozytywny 2: dodanie wielu produktow naraz
commit e0e4748427601d7f1a2d993a3a026145b6074a0a
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 13:18:12 2026 +0100
Test pozytywny 1: polaczenie koszyka z magazynem i dodanie do koszyka przedmiotu z magazynu
commit bfa19d20ff9bf5d50f40965f955b461fc32433e5
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 13:08:50 2026 +0100
Utworzenie pliku do testow integracyjnych - MagazynSklepuIntegracyjneTests.cs
commit db5b5281c50df76a113c3ec9995e0bb931e6e339
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 12:52:50 2026 +0100
dodanie metody PobierzCene do pobierania cen z magazynu
10°C
Pochmurnie
POL
100%
13:50
29.10.2026


C:\Windows\System32\cmd.e
[master db5b528] dodanie metody PobierzCene do pobierania cen z magazynu
1 file changed, 8 insertions(+)
C:\Users\tomek\Desktop\5P>git log
commit db5b5281c50df76a113c3ec9995e0bb931e6e339 (HEAD -> master)
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 12:52:50 2026 +0100
dodanie metody PobierzCene do pobierania cen z magazynu
commit 0cfebd5d50e27641bdb6e7ccd43a249c4faf1d21
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 12:43:53 2026 +0100
dodanie metody SprawdzDostepnosc z walidacja nazw i ilosci
commit 24fc1044772547791209caf1180b649f85becc260
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 12:32:56 2026 +0100
dodanie slownika ze stanem produktow
commit ee52fa2d480b40ab3cb2d111201d6de843b89651
Author: Piotr <pepe082822@gmail.com>
Date: Thu Oct 29 12:24:34 2026 +0100
dodanie modulu magazynu i stworzenie klasy MagazynSklepu
commit 03085eb6a9c9664cf01bcf92f2de70fc0c94e433 (Repo/master, Repo/HEAD)
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 18:42:07 2026 +0200
Dodanie sprawozdania do projektu 1 (sklep internetowy)
commit ff2a8c1b261e0fe7c0e36f5c0e610862021a9184
Author: Piotr <pepe082822@gmail.com>
Date: Tue Sep 29 18:18:48 2026 +0200
Dodanie komentarzy w testach
commit 0f41e46b1fae89f4f54409d17c0814874ba4312c
9°C
Duże zachmurze...
POL
100%
12:53
29.10.2026